

TECHNICAL UNIVERSITY

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #5

Trees and operations

Computer Science

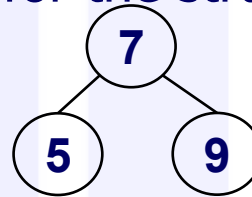
Agenda

- **Trees**
 - representation
 - basic operations
- **Tree traversal**
 - pre, in, post orders
- **Tree transformation into an ordered list**
 - Forward, backward, can we do better?
- **Trees (BST and regular trees)**
 - insert, search, delete

Trees

- Structure not defined; it is (used) as we define them
- Traditional definition is either prefix or infix notation
 - with a letter as name for the structure: *n* or *t*.

- Therefore, the tree



- Is represented in one of the forms:

prefix notation:

```
tree(t(7,t(5,nil,nil),t(9,nil,nil))).
```

infix notation:

```
tree(n(n(nil,5,nil),7,n(nil,9,nil))).
```

- Existing trees are represented as facts (predicate tree) in the program

Trees – operations

- Same as in other languages
- **Basic operations:**
 - Traversal
 - Search
 - Insert
 - Delete

Trees – traversal

- 3 types: pre/in/post order

```
inorder(nil) .
```

```
inorder(n(Left,Key,Right)) :-  
    inorder(Left) ,  
    do_something(Key) ,  
    inorder(Right) .
```

- The predicate has no outcome/output. Why?
 - No arguments!
- The order of clauses doesn't matter (due to indexation on the first argument)
- The predicate `do_something` does the actual job
- It runs $O(n)$ if `do_something` has constant time $O(1)$.
- Otherwise, the time is calculated with the Master Theorem.
- pre/post orders are similar, `do_something` goes first/last.

Tree transformation into an ordered list

- Given a Binary Search Tree (BST; what is it? Why do we represent trees as search trees? Benefits? Limitations? Means of overcoming limitation?)
- Generate the ordered list of the keys in the tree
- Approaches: divide et impera
 - divide – already done (the tree structure is a partition already)
 - impera – combine the results obtained on the subsets of the partition
 - Depending on how efficient we combine them we get the efficiency of the solution
- Approaches considered:
 - Forward and backward recursion
 - Pre/in/post order (which one is available in the context)

Computer Science

Tree 2 list forward

- By the approach, besides the output argument, we need a supplementary argument, some partial result to accumulate the results "so far"
- The arguments needed are:
 - input tree, accumulator (list type), output list
- Order of arguments:
 - input has to come first to benefit from the first argument indexation
- The other arguments are in any order
- The meaning of the accumulator = the elements from the tree we already covered placed in an ordered list

```
% inorder1/3 (Tree, Partial_result_list, Final_result_list)
```

Tree 2 list forward – code

```
inorder1(nil,L,L).  
inorder1(n(Left,Key,Right), Lin, Lout):-  
    inorder1(Left, Lin, LLout),  
    append(LLout,[Key],LRin),  
    inorder1(Right,LRin,Lout).
```

Clauses:

- When we reached the empty tree, the accumulator instantiates the result (default unification) as always in forward recursion.
- For the nonempty tree, the forward approach solves the parts of the partition in the standard order (first to last flow)
 - Solve the problem on the left subtree; the accumulator (content of the tree already covered, `Lin`) is passed as partial result. The result computed by this call will add all the keys in the left subtree to that accumulator and passes as accumulator (`LLout`) to the next processing task; `inorder1(Left,Lin,LLout)`,
 - Solve the problem on the next part in the partition = `Key`; `append(LLout,[Key],LRin)`,
 - Solve the problem on the right subtree; the accumulator (content of the tree already covered, `LRin`) is passed as partial result. The result computed by this call will add all the keys in the right subtree to that accumulator and passed as final result = on the whole tree); `inorder1(Right,LRin,Lout)`.

Tree 2 list forward – code analysis

```
inorder1 (nil, L, L) .
```

```
inorder1 (n (Left, Key, Right), Lin, Lout) :-  
    inorder1 (Left, Lin, LLout),  
    append (LLout, [Key], LRin),  
    inorder1 (Right, LRin, Lout) .
```

- Needs specific initial call (wrapper) with additional argument (accumulator):

```
inorder1 (Tree, Result) :-  
    inorder1 (Tree, [], Result) .
```

- Evaluation:

- 2 recursive calls ($a=2$)
- 2 proportional parts (assuming tree is balanced) ($b=2$)
- Outside recursive call linear time (append) ($c=1$)
- Hence, **$O(n \lg n)$** – supralinear time, but a small quantity over n
- Please come back to this code AFTER we study difference lists

Tree 2 list backward – code

- Needs no additional argument
- The arguments needed are:
 - input tree, output list
- Order of arguments:
 - input has to come first to benefit from the first argument indexation

```
% inorder2/2 (In_tree, Result_list)
```

```
inorder2(nil, []).
```

```
inorder2(n(Left,Key,Right),Lout):-
```

```
    inorder2(Left,LLout),
```

```
    append(LLout, [Key], Lintout),
```

```
    inorder2(Right,LRout),
```

```
    append(Lintout, LRout, Lout).
```

Tree 2 list backward – code

```
inorder2(nil, []).  
inorder2(n(Left, Key, Right), Lout) :-  
    inorder2(Left, LLout),  
    append(LLout, [Key], Lintout),  
    inorder2(Right, LRout),  
    append(Lintout, LRout, Lout).
```

Clauses:

- When we reached the empty tree, the result is empty.
- For the nonempty tree, the backward approach solves:
 - left subtree; the keys from left subtree generate the list from left out (LLout),
 - on the next part in the partition = Key to be added at the end of the LLout to create Lintout; `append(LLout, [Key], Lintout)`,
 - right subtree; the keys from right subtree generate the list from right out (LRout),
 - finally, the results of parts are concatenated: `append(Lintout, LRout, Lout)`.

Tree 2 list backward– code analysis

```
inorder2(nil, []).  
inorder2(n(Left, Key, Right), Lout) :-  
    inorder2(Left, LLout),  
    append(LLout, [Key], Lintout),  
    inorder2(Right, LRout),  
    append(Lintout, LRout, Lout).
```

- Evaluation
 - 2 recursive calls ($a=2$)
 - 2 proportional parts (assuming tree is balanced) ($b=2$)
 - Outside recursive call linear time (append) ($c=1$)
 - Hence, **$O(n \lg n)$** – supralinear time, but a small quantity over n
- Compared to the previous strategy?
 - Time is larger (2 append calls instead of 1)

Tree 2 list backward – better?

- What can be done to improve?
- 2 append calls, both traversing (almost) through the same list
- The first append goes over the whole first argument (large list) in order to add just one element (Key) at the end.
- Can we make it better? What if we rephrase:
 - Add at the end of the list with add at the beginning of a list. How?
 - Instead of adding `Key` at the end of `LLout`
 - Add it at the beginning of `LRout`
 - But `LRout` is NOT available at this moment
 - Change order of processing

Computer Science

Tree 2 list backward – code

```
inorder2(nil, []).  
inorder2(n(Left, Key, Right), Lout) :-  
    inorder2(Left, LLout),  
    append(LLout, [Key], Lintout),  
    inorder2(Right, LRout),  
    append(Lintout, LRout, Lout).
```

```
inorder3(nil, []).  
inorder3(n(Left, Key, Right), Lout) :-  
    inorder3(Left, LLout),  
    inorder3(Right, LRout),  
    append(LLout, [Key | LRout], Lout).
```

Clauses:

- When we reached the empty tree, the result is empty.
- For the nonempty tree, the backward approach solves:
 - left subtree; the keys from left subtree generate the list from left out (LLout),
 - right subtree; the keys from right subtree generate the list from right out (LRout),
 - finally, the results of parts are concatenated with the unit element in between (before the result on the right): `append(LLout, LRout, Lout)`.

Tree 2 list backward– code analysis

```
inorder3(nil, []).  
inorder3(n(Left, Key, Right), Lout) :-  
    inorder3(Left, LLout),  
    inorder3(Right, LRout),  
    append(LLout, [Key| LRout], Lout).
```

- Evaluation
 - 2 recursive calls ($a=2$)
 - 2 proportional parts (assuming tree is balanced) ($b=2$)
 - Outside recursive call linear time (append) ($c=1$)
 - Hence, **$O(n \lg n)$**
- Better
 - than the previous backward as just one append (multiplicative constant)
 - than forward – needs no additional parameter
- Can we do even better? Answer – we can! Check after difference lists

Tree 2 list conclusions (so far)

- Neither (fwd or bwd) is optimal (both are supralinear)
- Both suffer from the additional time with `append/3`
- How can we handle that? TBD soon

Insert in BST

```
% insert_bst/3(In_tree, Key, Out_tree).
```

Q: is the order of arguments relevant ? Why/why not?

```
insert_bst(nil, K, n(nil,K,nil)):-!. % insert_bst(T, K, R):-T=nil,!, R = t(nil,K,nil).
insert_bst(n(Left,K,Right), K, n(Left,K,Right)):-!,
    write("in tree").
insert_bst(n(Left,Key,Right), K, n(NewLeft,Key,Right)):-
    K<Key,!,
    insert_bst(Left,K,NewLeft).
insert_bst(n(Left,Key,Right), K, n(Left,Key,NewRight)):-
    insert_bst(Right,K,NewRight).
```

Qs:

1. Where is inserted (position)? ALWAYS LEAF! NO EXCEPTION!
2. What does ! in clause 1 negate? In clause 2?

Time estimate: a=1, b=2 (if balanced), c=0 =>O(lgn)

Q: Estimate best/worst case (situation) and time

Insert in regular (not search) tree

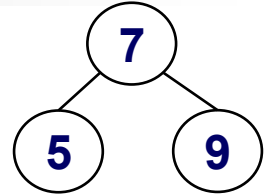
```
% insert_tree/3(In_tree, Key, Out_tree).  
  
insert_tree(nil, K, n(nil,K,nil)):-!. % insert_tree(Tin, K, Tout):-Tin=nil, !, Tout=t(nil,Key,nil).  
insert_tree(n(Left,K,Right), K, n(Left,K,Right)):- % insert_tree(Tin,K,Tout):-  
    !, write("in tree"). % Tin=t(Left,K,Right),!,write("in tree"),Tout= t(Left,K,Right).  
insert_tree(n(Left,Key,Right), K, n(NewLeft,Key,Right)):-  
    insert_tree(Left, K, NewLeft).  
insert_tree(n(Left,Key,Right), K, n(Left,Key,NewRight)):-  
    insert_tree(Right, K, NewRight).
```

- **Qs:**
 - **1.** Where is inserted (position)?

Search in BST (search Node by Key)

Tree stored as a fact `tree/1` in knowledge base

infix notation: `tree(n(n(nil, 5, nil), 7, n(nil, 9, nil)))`.



```

% search_bst/2 (Key, Tree)

search_bst(Key, n(_, Key, _)).
search_bst(Key, n(L, K, _)):-
    Key < K, !,
    search_tree(Key, L).
search_bst(Key, n(_, _, R)):-
    search_bst(Key, R).

?- tree(T), search_bst(5, T).
true, T=...
  
```

Computer Science

Search in BST

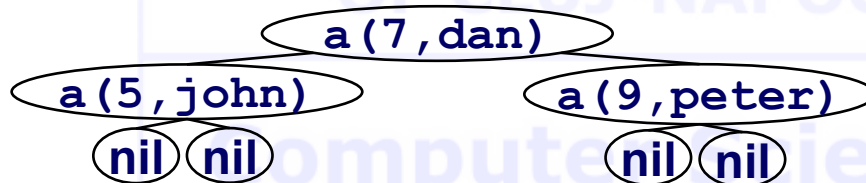
Assume the node contains a structure of type `a(Key,Value)`

The tree is BST based on key

Ex:

```
tree(
  n(
    n(nil, a(5, john), nil),
    a(7, dan),
    n(nil, a(9, peter), nil)
  )
).
```

Represents a tree with 7 and 2 leaves (5 and 9).



If `tree/1` is stored in the knowledge base, processing can be done with:

```
?- tree(T), process(T, ...).
```

Search in BST (search Node by Key)

```
%search_bst/2 (Node,Tree)
```

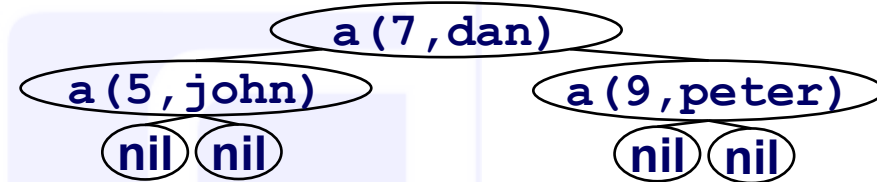
```
search_bst(N, n(_,Node,_)):-  
    eq(N, Node),!,  
    N=Node.
```

```
search_bst(N, n(Left,Node,_)):-  
    ord(N, Node),!,  
    search_bst(N, Left).
```

```
search_bst(N, n(_,_,Right)):-  
    search_bst(N, Right).
```

```
eq(a(K,_),a(Key,_)):-  
    nonvar(K),  
    K=Key,!.
```

```
ord(a(K,_),a(Key,_)):-  
    nonvar(K),  
    K<Key,!.
```



Search in BST (search Node by Key)

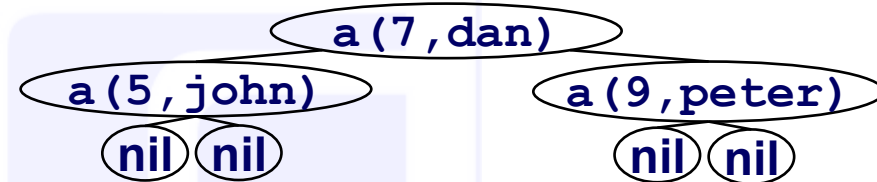
```
eq(a(K, _), a(Key, _)) :-  
    nonvar(K),  
    K=Key, !.
```

- Predicate to check if the *key* we are *searching* for has the **same** value as the *key* in the *current node* in the tree.
- Qs:
 - What is the meaning of `nonvar(K)` and why is needed?
 - "!" is not mandatory! Why?

```
ord(a(K, _), a(Key, _)) :-  
    nonvar(K),  
    K<Key, !.
```

- Predicate to check if the key *K* we are *searching* for has a **smaller** value than the key *Key* in the *current node* in the tree.
- Qs:
 - What is the meaning of `nonvar(K)` and why is needed?
 - "!" is not mandatory! Why?

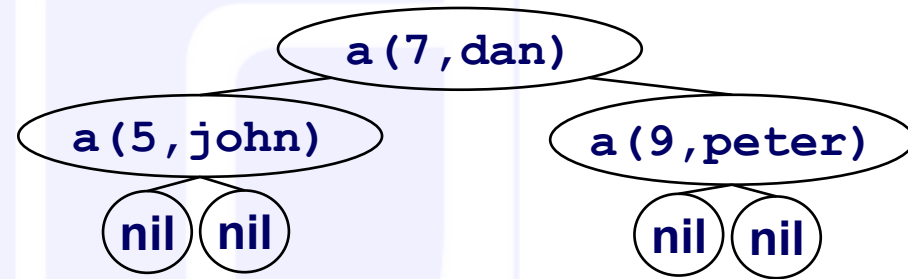
```
?- tree(T), search_bst(a(9, X), T).  
true, T=..., X=peter.
```



Search in regular tree (search Node by Value)

```
% search_tree/2 (Key, Tree)

search_tree(N, n(_, N, _)).
search_tree(N, n(Left, _, _)) :-
    search_tree(N, Left).
search_tree(N, n(_, _, Right)) :-
    search_tree(N, Right).
```



```
?- tree(T), search_tree(a(R, john), T).
true, T=..., R=5.
```

% T would be the tree read, R the key assoc. to the value (name) provided

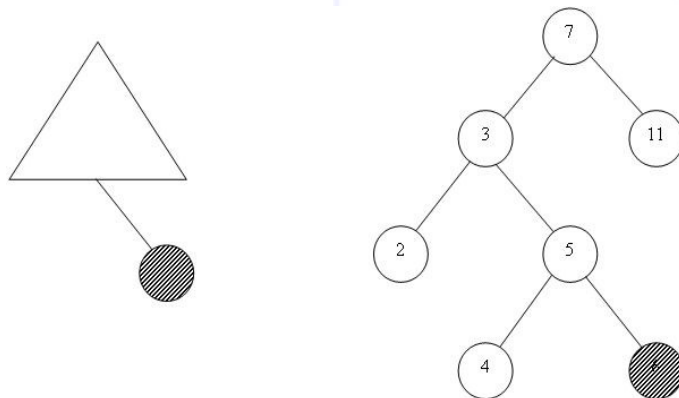
```
?- tree(T), search_tree(a(9, X), T).
```

% can we ask this way? What's the answer? What's the meaning?
true, T=..., X=peter

Delete from BST

- Search for the node with that key + remove the node
- Delete a key = remove the node containing the key
- Cases to consider:
 - Leaf – just remove the node
 - One child node – shortcut the node (link the child to its grandparent)
 - Two children ($\Leftrightarrow$ delete the root after the search stage) different story

Case 1



Case 2

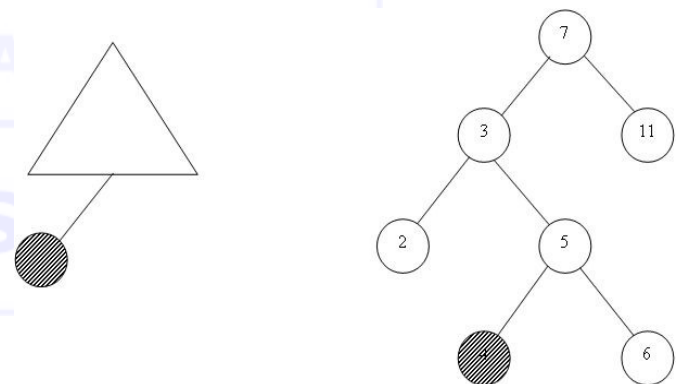
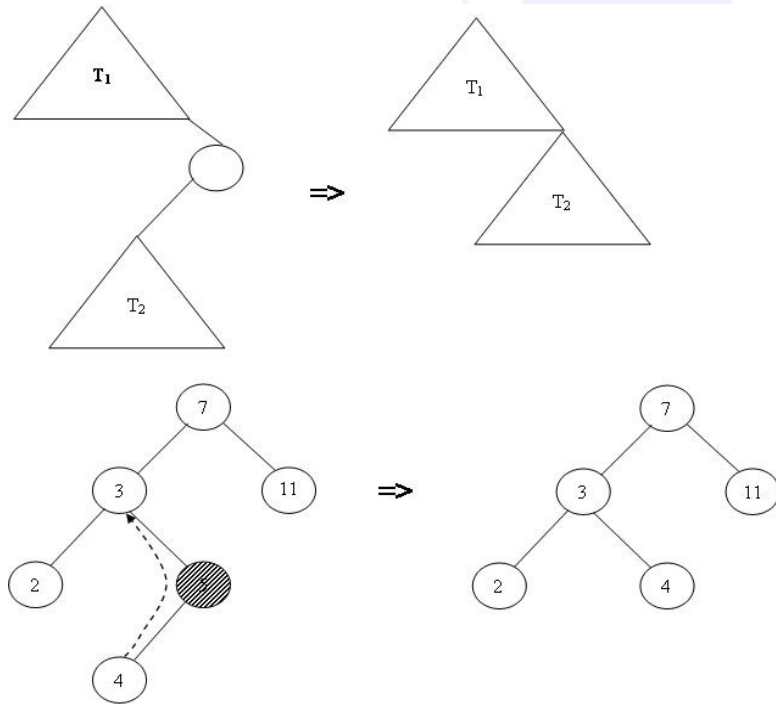


Figure. Removal of leaves

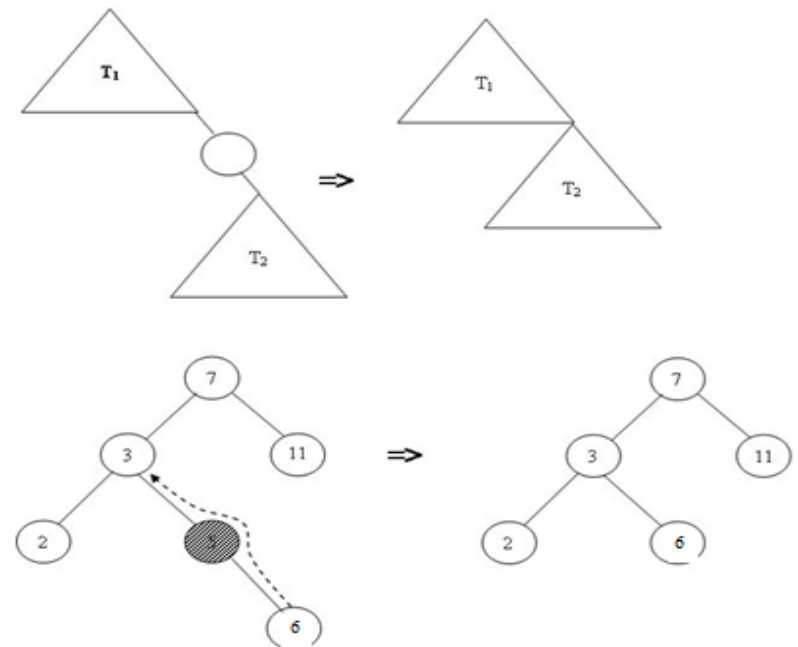
Delete from BST

Removal of a right child node with just one subtree

Case 3

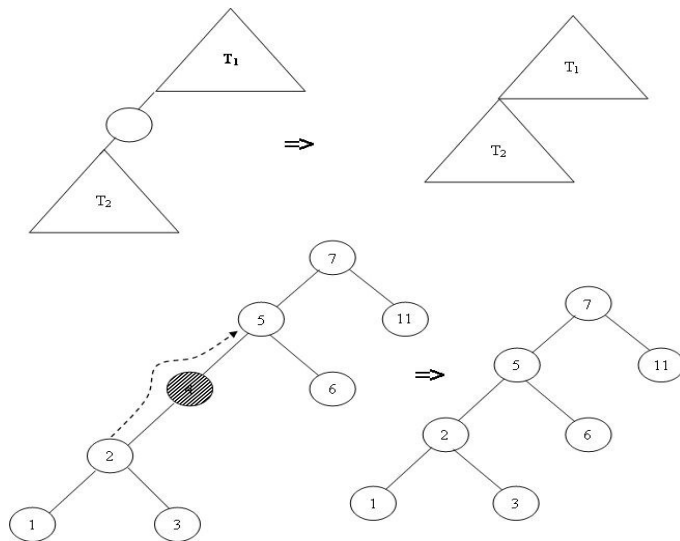


Case 4



Delete from BST

Case 5



Case 6

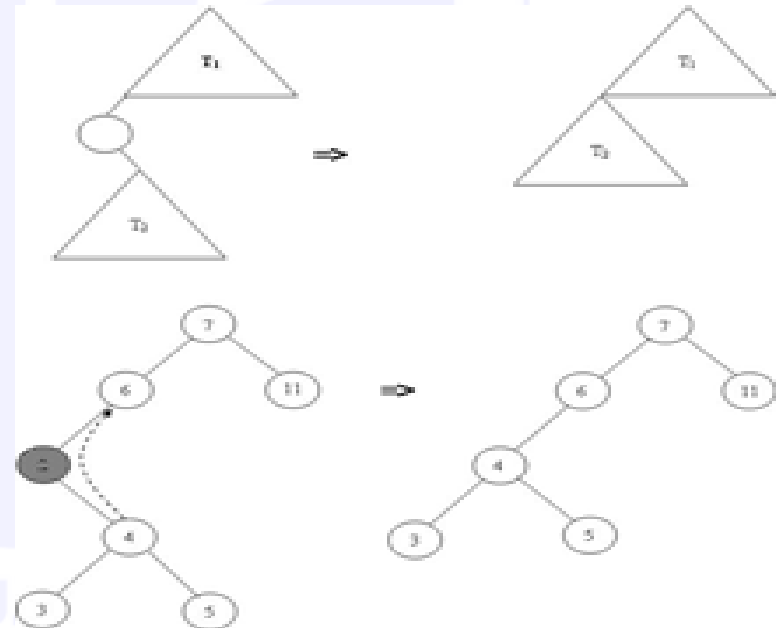


Figure. Removal of a left child node with just one subtree

- How about root? Postpone for the moment.
- Let's reach this point with code.

Delete from BST - code

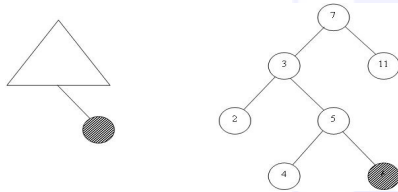
```
% delete_bst/3(In_tree, Key_to_del, Out_tree)

% search key; find the node to contain the key
delete_bst(n(Left,Key,Right),K,n(NewLeft,Key,Right)):-
    K<Key,!,
    delete_bst(Left,K,NewLeft).
delete_bst(n(Left,Key,Right),K,n(Left,Key,NewRight)):-
    K>Key,!,
    delete_bst(Right,Key,NewRight).

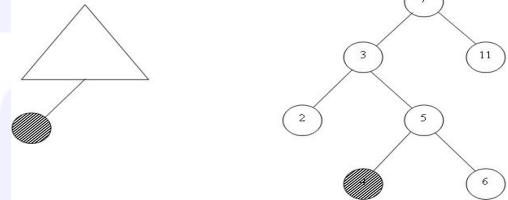
% found key; let's solve the easy cases discussed before
delete_bst(nil, K, nil):-!,
    write(K), write('not in tree').
delete_bst(n(nil, K, nil), K, nil):-!.
delete_bst(n(nil, K, Right), K, Right):-!.
delete_bst(n(Left, K, nil), K, Left):-!.

% "difficult" case postponed
```

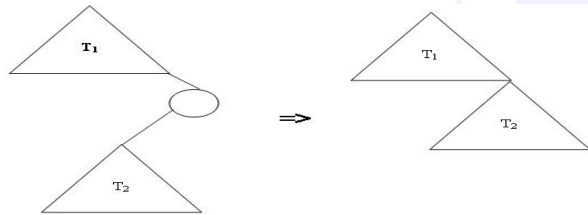
Delete cases



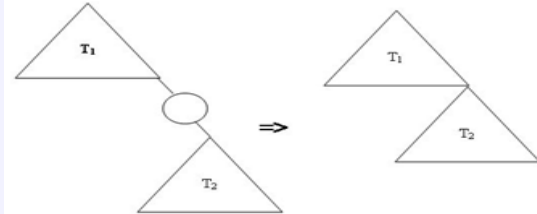
1



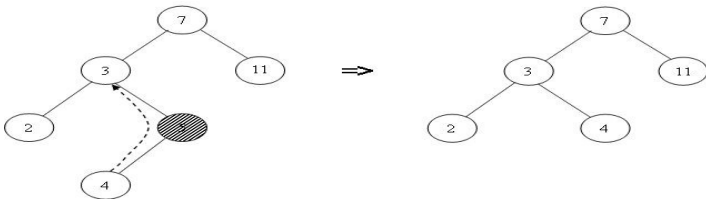
2



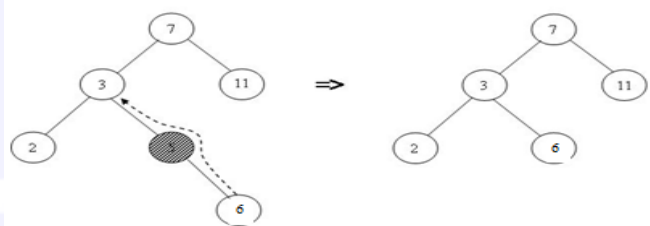
3



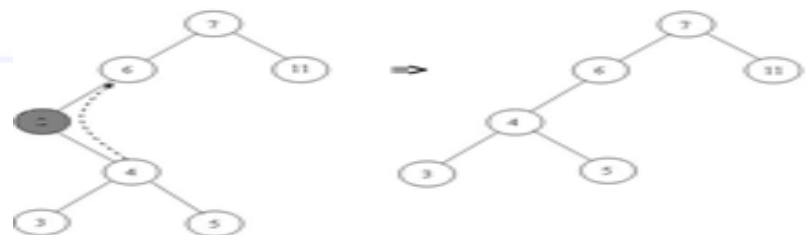
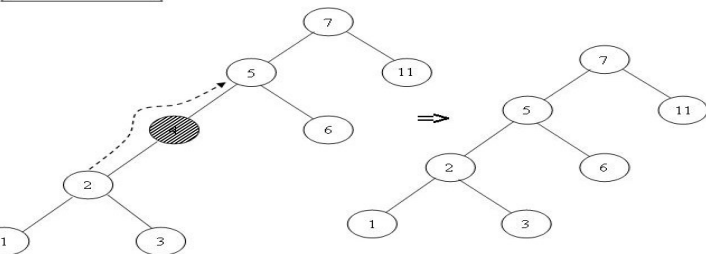
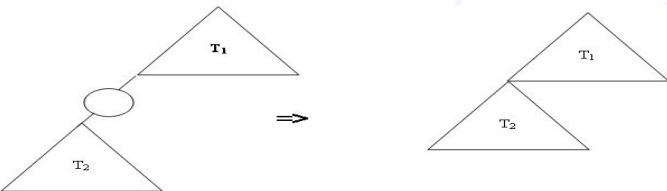
4



5



6



Delete from BST - code

```
% delete_bst/3(In_tree, Key_to_del, Out_tree)
[d1] delete_bst(n(Left,Key,Right), K, n(NewLeft,Key,Right)):-
    K<Key,!,
    delete_bst(Left, K, NewLeft).
[d2] delete_bst(n(Left,Key,Right), K, n(Left,Key,NewRight)):-
    K>Key,!,
    delete_bst(Right, K, NewRight).

[d3] delete_bst(nil, K, nil):-!,
    write(K),write('not found').
[d4] delete_bst(n(nil,K,nil), K, nil):-!.
[d5] delete_bst(n(nil,K,Right), K, Right):-!.
[d6] delete_bst(n(Left,K,nil), K, Left):-!.
```

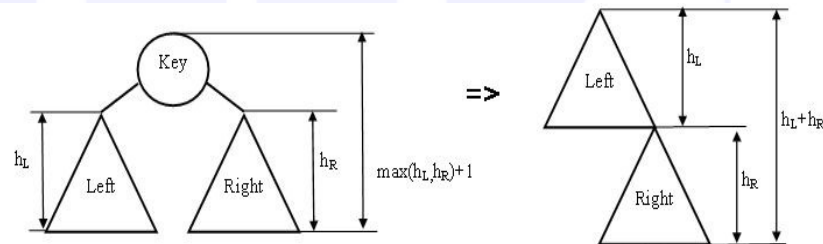
Qs:

- What cases did we cover so far?
- Which clause covers which case? Explain!
- Do we need all clauses so far? Why/why not? Justify

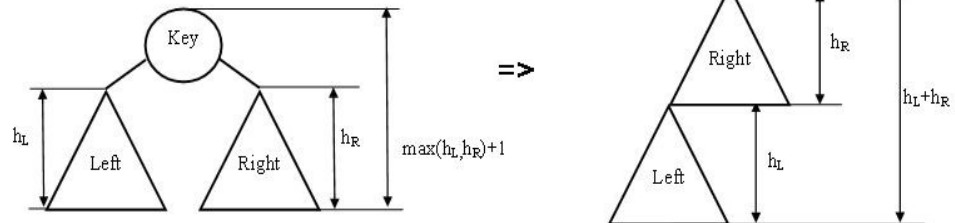
Delete from BST – code

- We reached the node containing the key to delete
- Has 2 children = is the root of a subtree =>
 - problem becomes to remove the root of the tree
- How to...? We have alternatives:
 - Either link the subtrees:
 - Easy to apply (time estimate)
 - Disadvantage?

Sol1



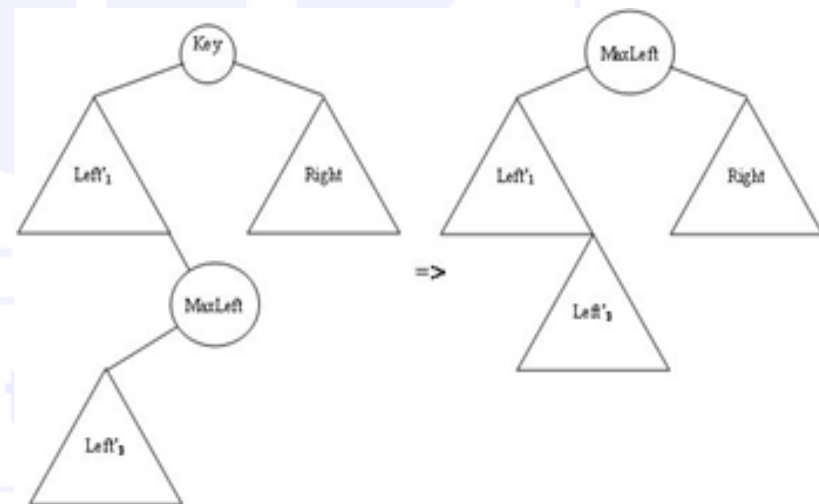
Sol2



Delete from BST – code

- Or replace problem to solve with a problem you know how to solve!
- Replace **INSTEAD** the node you want to remove (the root) with another one. How:
 - Find an easy to remove node
 - Place it instead of the root (is possible?)
 - To be possible, that node should be a node in inorder:
 - Either before the key in the root
 - Or after the key in the root
 - So, replace with
 - Predecessor
 - Successor
 - !
 - Are those nodes with the desired property?
 - Justify

Sol3



Delete from BST – code

```
delete_bst(n(Left,Key,Right), Key, n(NewLeft,MaxLeft,Right)):-!,  
    deleteMaxFromTree(Left,MaxLeft,NewLeft).
```

```
% deleteMaxFromTree/3(In_tree,Max_key,Out_Tree).
```

- The predicate takes a tree as input `In_tree`, finds the max key `Max_key` (the rightmost node) and removes it from the tree, returning the tree `Out_tree` without that key.

```
deleteMaxFromTree(n(Left,Key,Right), MaxKey, n(Left,Key,NewRight)):-  
    deleteMaxFromTree(Right,MaxKey,NewRight).  
deleteMaxFromTree(n(Left,MaxKey,nil), MaxKey, Left):-!.
```

- Clause 1 says: as long as the right subtree is not empty, go to the right
- Clause 2 says: if right is `nil`, you found the max key `MaxKey` and `Left` is out tree.
- Do we ever reach second clause? Infinite loop. Clauses are not in the correct order! Let's change it!

Delete from BST – code

```
deleteMaxFromTree (n (Left, MaxKey, nil), MaxKey, Left) :- !.  
deleteMaxFromTree (n (Left, Key, Right), MaxKey, n (Left, Key, NewRight)) :-  
    deleteMaxFromTree (Right, MaxKey, NewRight) .
```

- It is inefficient. We always try the non-useful clause first
- Does it really need to have the fact first?
- No, it can go second (JUSTIFY!).

```
deleteMaxFromTree (n (Left, Key, Right), MaxKey, n (Left, Key, NewRight)) :-  
    deleteMaxFromTree (Right, MaxKey, NewRight) .  
deleteMaxFromTree (n (Left, MaxKey, nil), MaxKey, Left) :- !.
```

Computer Science

Delete from BST – complete code

% search

```
delete_bst(n(Left,Key,Right), K, n(NewLeft,Key,Right)):-  
    K<Key, !,  
    delete_bst(Left, K, NewLeft).  
delete_bst(n(Left,Key,Right), K, n(Left,Key,NewRight)):-  
    K>Key, !,  
    delete_bst(Right, K, NewRight).
```

% cases of no children or 1 child

```
delete_bst(nil, K, nil):- !, write(K), write('not found').  
delete_bst(n(nil,K,nil), K, nil):- !.  
delete_bst(n(nil,K,Right), K, Right):- !.  
delete_bst(n(Left,K,nil), K, Left):- !.
```

% case of 2 children (starts search from predecessor on Left)

```
delete_bst(n(Left,Key,Right), Key, n(NewLeft,MaxLeft,Right)):-!,  
    deleteMaxFromTree(Left,MaxLeft,NewLeft).
```

% search for max (always go on the Right, stop when nil is reached)

```
deleteMaxFromTree(n(Left,Key,Right),MaxKey,n(Left,Key,NewRight)):-  
    deleteMaxFromTree(Right,MaxKey,NewRight).  
deleteMaxFromTree(n(Left,MaxKey,nil),MaxKey,Left):-!.
```

Delete from BST – code analysis

- Search phase: $O(h)$
- Delete (no matter the solution/case chosen): $O(h)$
- Overall: $2h$
- $O(h)$, constant 2. Is it so?
- It is just $O(h)$, constant 1. JUSTIFY!